25.Question 2: Decision Tree for Iris Flower Classification You are analyzing the famous Iris flower dataset to classify iris flowers into three species based on their sepal and petal dimensions. You want to use a Decision Tree classifier to accomplish this task. Write a Python program that loads the Iris dataset from scikit-learn, and allows the user to input the sepal length, sepal width, petal length, and petal width of a new flower. The program should then use the Decision Tree classifier to predict the species of the new flower.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
iris = load_iris()
X = iris.data
y = iris.target
model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)
sl = float(input("Enter sepal length: "))
sw = float(input("Enter sepal width: "))
pl = float(input("Enter petal length: "))
pw = float(input("Enter petal width: "))
flower = [[sl, sw, pl, pw]]
prediction = model.predict(flower)
print("Predicted Iris Species:", iris.target_names[prediction[0]])
```

```
Enter sepal length: 2.3
Enter sepal width: 2
Enter petal length: 1.2
Enter petal width: 2.5
Predicted Iris Species: setosa
```

26.Question 3: Linear Regression for Housing Price Prediction You are a real estate analyst trying to predict housing prices based on various features of the houses, such as area, number of bedrooms, and location. You have collected a dataset of houses with their respective prices. Write a Python program that allows the user to input the features (area, number of bedrooms, etc.) of a new house. The program should use linear regression from scikit-learn to predict the price of the new house based on the input features.

```
from sklearn.linear_model import LinearRegression
X = [[1000, 2],[1500, 3],[2000, 3],[2500, 4],[3000, 4],
[3500, 5]]
y = [2000000, 3000000, 4000000, 5000000, 6000000, 7000000]
model = LinearRegression()
model.fit(X, y)
area = float(input("Enter house area in sq. ft.: "))
bedrooms = int(input("Enter number of bedrooms: "))
house = [[area, bedrooms]]
price = model.predict(house)
print("Predicted House Price: Rs.", round(price[0], 2))
```

```
Enter house area in sq. ft.: 2600
Enter number of bedrooms: 7
Predicted House Price: Rs. 5200000.0
```

27.Question: Logistic Regression for Customer Churn PredictionYou are working for a telecommunications company, and you want to predict whether a customer will churn (leave the company) based on their usage patterns and demographic data. You have collected a dataset of past customers with their churn status (0 for not churned, 1 for churned) and various features. Write a Python program that allows the user to input the features (e.g., usage minutes, contract duration) of a new customer. The program should use logistic regression from scikit-learn to predict whether the new customer will churn or not based on the input features.

```
from sklearn.linear_model import LogisticRegression
X = [[100, 12],[200, 24],[50, 6],[300, 36],
[80, 6],[250, 24],[60, 3],[180, 12]]
y = [1, 0, 1, 0, 1, 0, 1, 0]
model = LogisticRegression()
model.fit(X, y)
minutes = float(input("Enter monthly usage minutes: "))
duration = int(input("Enter contract duration in months: "))
customer = [[minutes, duration]]
prediction = model.predict(customer)
if prediction[0] == 1:
    print("Customer is likely to churn.")
else:
    print("Customer is not likely to churn.")
```

```
Enter monthly usage minutes: 50
Enter contract duration in months: 5
Customer is likely to churn.
```

28.Question: K-Means Clustering for Customer Segmentation You are working for an e-commerce company and want to segment your customers into distinct groups based on their purchasing behavior. You have collected a dataset of customer data with various shopping-related features.Write a Python program that allows the user to input the shopping-related features of a new customer. The program should use K-Means clustering from scikit-learn to assign the new customer to one of the existing segments based on the input features.

```
from sklearn.cluster import KMeans
X = [[1000, 2],[1200, 3],[1500, 2],
[8000, 10],[9000, 12],[8500, 11],
[3000, 5],[3500, 4]]
model = KMeans(n_clusters=3, random_state=42, n_init=10)
model.fit(X)
amount = float(input("Enter monthly purchase amount: "))
frequency = int(input("Enter number of monthly purchases: "))
customer = [[amount, frequency]]
segment = model.predict(customer)
print("Customer belongs to Segment:", segment[0])
```

```
Enter monthly purchase amount: 560000
Enter number of monthly purchases: 7
Customer belongs to Segment: 1
```

29.Question: Evaluation Metrics for Model Performance You have trained a machine learning model on a dataset, and now you want to evaluate its performance using various metrics. Write a Python program that loads a dataset and trained model from scikit-learn. The program should ask the user to input the names of the features and the target variable they want to use for evaluation. The program should then calculate and display common evaluation metrics such as accuracy, precision, recall, and F1-score for the model's predictions on the test data.

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
data = load_breast_cancer(as_frame=True)
df = data.frame
print("Available features:")
print(list(data.feature_names))
print("Target variable: target")
features = input("Enter feature names separated by commas: ").split(",")
features = [f.strip() for f in features]
target = input("Enter target variable name: ").strip()
X = df[features]
y = df[target]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1-Score:", f1_score(y_test, y_pred, zero_division=0))
```

```
Available features:
[np.str_('mean radius'), np.str_('mean texture'), np.str_('mean perimeter'), np.str_('mean area'), np.str_('mean smoothness'), np.str_('mean
Target variable: target
Enter feature names separated by commas: mean radius,mean texture
Enter target variable name: target
Accuracy: 0.8771929824561403
Precision: 0.9384615384615385
Recall: 0.8591549295774648
F1-Score: 0.8970588235294118
```

30.Question: Classification and Regression Trees (CART) for Car Price Prediction You are working for a car dealership, and you want to predict the price of used cars based on various features such as the car's mileage, age, brand, and engine type. You have collected a dataset of used cars with their respective prices. Write a Python program that loads the car dataset and allows the user to input the features of a new car they want to sell. The program should use the Classification and Regression Trees (CART) algorithm from scikit-learn to predict the price of the new car based on the input features.The CART algorithm will create a tree-based model that will split the data into subsets based on the chosen features and their values, leading to a decision path that eventually predicts the price of the car. The program should output the predicted price and display the decision path (the sequence of conditions leading to the prediction) for the new car.

```
    feature_names=["Mileage", "Age", "Brand", "Engine"])
print("Decision Tree Rules:")
print(tree_rules)
print("Decision Path for the Car:")
node_indicator = decision_path.indices[
    decision_path.indptr[0]:decision_path.indptr[1]]
tree = model.tree_
for node_id in node_indicator:
    if tree.children_left[node_id] != tree.children_right[node_id]:
        feature = tree.feature[node_id]
        threshold = tree.threshold[node_id]
        name = ["Mileage", "Age", "Brand", "Engine"][feature]
        if car[0][feature] <= threshold:
            print(name, "<=", round(threshold, 2))
        else:
            print(name, ">", round(threshold, 2))
print("Reached prediction leaf.")
```

```
Enter car mileage: 765
Enter car age in years: 3
Enter brand code (1-4): 4
Enter engine type (0-Petrol, 1-Diesel): 0
Predicted Car Price: Rs. 800000.0
Decision Tree Rules:
|--- Mileage <= 55000.00
|   |--- Mileage <= 35000.00
|   |   |--- Engine <= 0.50
|   |   |   |--- value: [800000.00]
|   |   |--- Engine >  0.50
|   |   |   |--- value: [900000.00]
|   |--- Mileage >  35000.00
|   |   |--- Mileage <= 45000.00
|   |   |   |--- value: [700000.00]
|   |   |--- Mileage >  45000.00
|   |   |   |--- value: [600000.00]
|--- Mileage >  55000.00
|   |--- Age <= 7.50
|   |   |--- Engine <= 0.50
|   |   |   |--- value: [400000.00]
|   |   |--- Engine >  0.50
|   |   |   |--- value: [500000.00]
|   |--- Age >  7.50
|   |   |--- Age <= 8.50
|   |   |   |--- value: [300000.00]
|   |   |--- Age >  8.50
|   |   |   |--- value: [200000.00]
```

```
Decision Path for the Car:
Mileage <= 55000.0
Mileage <= 35000.0
Engine <= 0.5
Reached prediction leaf.
```